รายงานสัปดาห์ที 11 (Basic)
Automating PDFs & Word Documents

สรุปเนือหา บันทึกผลแล็บ การแก้บัก 5 จุด และผลการรันจริง
โปรเจกต์: document-automator (PyPDF2 + python-docx)

---

1. ผลลัพธ์การเรียนรู้

- เข้าใจโครงสร้างของไฟล์ PDF และ Word ในมุมของการเข้าถึงด้วยโปรแกรม
- ใช้ PyPDF2 ดึงข้อความ รวมไฟล์ และแยกไฟล์ PDF
- ใช้ `python-docx` สร้างเอกสาร Word พร้อมหัวข้อ ย่อหน้า การจัดรูปแบบ และตาราง
- ออกแบบโปรแกรมแบบโมดูล แยกตรรกะ PDF และ Word ออกจากกัน
- จัดการข้อผิดพลาดและทำ logging ให้ตรวจสอบการทำงานย้อนหลังได้

2. สรุปเนือหาบทเรียน

2.1 การทำงานกับ PDF ด้วย PyPDF2

- ดึงข้อความ: `PdfReader(path)` แล้ววนอ่าน `page.extract_text()` ทีละหน้า
- รวมไฟล์: สร้าง `PdfWriter()` แล้ว `add_page()` ทีละหน้าจากไฟล์ต้นทางหลายไฟล์
- แยกไฟล์: อ่านหน้าทีละช่วงแล้วเขียนออกเป็นไฟล์ย่อย
- ข้อจำกัดสำคัญ: ดึงข้อความได้เฉพาะ PDF ทีฝังข้อความไว้จริง
  ถ้าเป็นไฟล์สแกนเป็นภาพจะได้ข้อความว่าง ต้องใช้ OCR แทน

2.2 การสร้างเอกสาร Word ด้วย python-docx

- `Document()` สร้างเอกสารเปล่า และ `add_heading(text, level)` ใส่หัวข้อ
- `add_paragraph()` ใส่ย่อหน้า จัดรูปแบบผ่าน `run.bold`, `run.italic`, `run.font.size`
- `add_table(rows, cols)` สร้างตาราง เข้าถึงช่องด้วย `table.cell(r, c)` และต้องกำหนด `table.style = 'Table Grid'` เพือให้มีเส้นตาราง
- แนวคิด run: ย่อหน้าหนึงประกอบด้วย run หลายชิน การจัดรูปแบบตัวอักษรทำทีระดับ run ส่วนการจัดวาง (alignment) ทำทีระดับย่อหน้า

3. สรุปผลการทำแล็บ

โปรแกรมทำงาน 2 งานต่อเนืองกัน งานแรกคือดึงข้อความจาก PDF มาสร้างเป็นรายงาน Word งานทีสองคือรวมไฟล์ PDF หลายไฟล์เข้าด้วยกัน

Task	สิงทีทำ	ผลลัพธ์
1. PDF → Word	ดึงข้อความจาก input_article.pdf (2 หน้า) แล้วสร้างเอกสาร Word ทีมีหัวข้อ ย่อหน้าตัวหนา ย่อหน้าตัวเอียงจัดกลาง ข้อความจาก PDF หัวข้อย่อย และตารางสรุป	output_report.docx

2. รวม PDF	นำ dummy_doc1.pdf และ dummy_doc2.pdf มาต่อกันเป็นไฟล์เดียว	merged_output.pdf (2 หน้า)

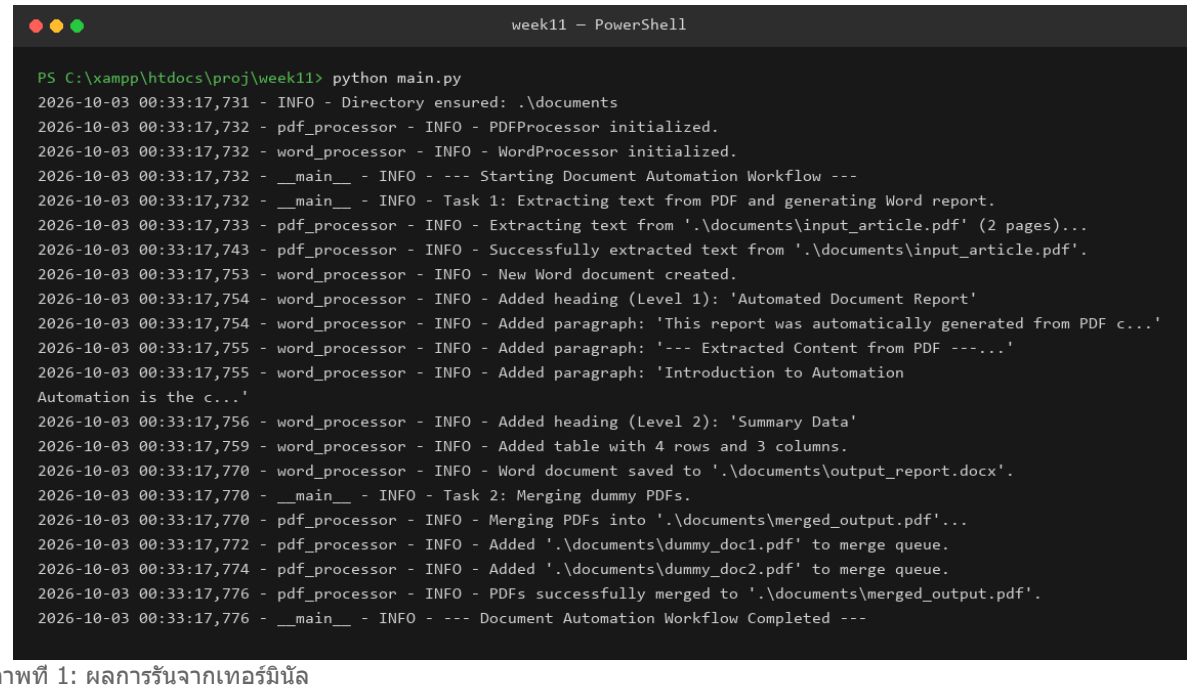
4. บั๊กที่พบในโค้ดต้นฉบับและการแก้ไข

พบปัญหา 5 จุด โดยจุดแรกร้ายแรงที่สุดเพราะเป็น syntax error ที่ทำให้ไฟล์ทั้งไฟล์ import ไม่ได้เลย

#	ไฟล์	อาการ	สาเหตุและการแก้ไข
1	pdf_processor.py	SyntaxError: invalid decimal literal — ไฟล์ import ไม่ได้ทั้งไฟล์	เมธอด split_pdf เขียน range(0_int, total_pages, ...) ซึ่ง 0_int ไม่ใช่ไวยากรณ์ Python (น่าจะพิมพ์ตกจาก int(0) หรือ 0) → แก้เป็น range(0, ...)
2	pdf_processor.py	NameError: ensure_directory_exists is not defined	split_pdf เรียกฟังก์ชันนี้แต่บรรทัด import ดึงมาเฉพาะ setup_logging → เพิ่มเข้าไปในบรรทัด import
3	pdf_processor.py word_processor.py	ImportError: attempted relative import	from .utils import ใช้ไม่ได้เพราะ main.py โหลดโมดูลผ่าน sys.path.append('src') ทำให้เป็น top-level module → เปลี่ยนเป็น absolute import
4	main.py	โค้ดเปราะบาง: datetime ถูก import ในบล็อก if __name__	ถ้ามีไฟล์อื่น import main.py แล้วเรียก main() จะเกิด NameError ทันทีเพราะ datetime ไม่เคยถูก import → ย้ายขึ้นไปไว้บนสุดของไฟล์
5	utils.py	ข้อความ log ถูกพิมพ์ซ้ำ 2 บรรทัด	basicConfig ใส่ handler ที่ root แล้ว setup_logging ใส่อีกที logger ลูก เมื่อ propagate ขึ้นไปจึงพิมพ์ซ้ำ → เพิ่ม logger.propagate = False

5. ผลการรันจริง (Test Results)

หลังแก้ครบทั้ง 5 จุด โปรแกรมทำงานจบสมบูรณ์ ทั้งสอง task สำเร็จ



ภาพที่ 1: ผลการรันจากเทอร์มินัล

5.1 ผลลัพธ์ที่ 1 — output_report.docx

output_report.docx

Automated Document Report

This report was automatically generated from PDF content and structured data.

--- Extracted Content from PDF ---

Introduction to Automation Automation is the creation and application of technology to produce and deliver goods and services with minimal human intervention. The implementation of automation technologies, techniques, and processes improves the efficiency, reliability, and speed of many tasks that were previously performed manually. This leads to increased productivity, reduced costs, and improved quality. --- Page End --- Benefits of Automation 1. Increased Efficiency: Tasks are completed faster and with higher throughput. 2. Cost Reduction: Lower labor costs and reduced waste. 3. Improved Accuracy: Minimizes human error. 4. Enhanced Safety: Dangerous tasks can be automated. 5. Greater Consistency: Uniform quality in output. Conclusion Automation is transforming industries globally. Embracing these technologies is crucial for future competitiveness. --- Page End ---

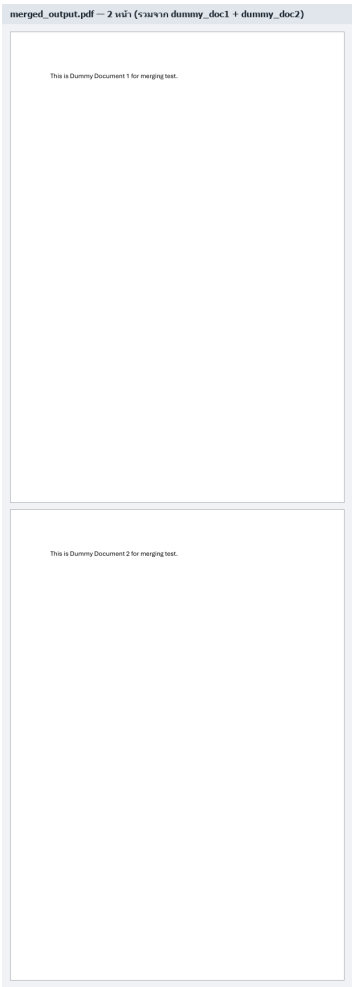
Summary Data

Metric	Value	Notes
Total Pages Processed	2	From Input PDF
Document Type	PDF/Word Hybrid	
Generated Date	2026-10-03	

ภาพที 2: เอกสาร Word ทีโปรแกรมสร้าง มีครบทังหัวข้อหลัก ย่อหน้าตัวหนา ย่อหน้าตัวเอียงจัดกลาง ข้อความทีดึงมาจาก PDF หัวข้อย่อย และตารางสรุป

ตรวจสอบความถูกต้อง: ค่า Total Pages Processed = 2 มาจากการนับตัวคัน `--- Page End ---` ในข้อความทีดึงมา ซึงตรงกับจำนวนหน้าจริงของ input_article.pdf ทีมี 2 หน้า และจะเห็นตัวคันนีปรากฏ 2 ครังในเนือหาทีแสดงในภาพ

5.2 ผลลัพธ์ที 2 — merged_output.pdf



ภาพที่ 3: PDF ที่รวมแล้ว ได้ 2 หน้าตามจำนวนไฟล์ต้นทาง หน้าแรกมาจาก dummy_doc1.pdf และหน้าที่สองมาจาก dummy_doc2.pdf

6. คำถาม-คำตอบสำคัญ

6.1 ทำไมบางครั้ง extract_text() ถึงได้ข้อความว่างเปล่า?

เพราะ PDF มีสองแบบ แบบแรกคือ PDF ที่สร้างจากโปรแกรมโดยตรง (เช่น Word บันทึกเป็น PDF) ซึ่งฝังข้อความไว้จริงและดึงออกมาได้ กับแบบที่สองคือ PDF ที่ได้จากการสแกนเอกสาร ซึ่งข้างในเป็นเพียงภาพไม่มีข้อความให้ดึง กรณีหลังต้องใช้ OCR เช่น `pytesseract` มาอ่านตัวอักษรจากภาพก่อน

6.2 ความแตกต่างระหว่าง paragraph กับ run ใน python-docx

Paragraph คือย่อหน้าหนึ่งย่อหน้า ส่วน Run คือช่วงข้อความย่อยภายในย่อหน้าที่มีรูปแบบเดียวกัน เช่นย่อหน้า "ราคา 1,200 บาท" จะมี 3 run (ปกติ / ตัวหนา / ปกติ) การตั้งตัวหนา ตัวเอียง หรือขนาดฟอนต์ ต้องทำที่ระดับ run ส่วนการจัดวางซ้าย-กลาง-ขวา ทำที่ระดับ paragraph ซึ่งเป็นเหตุผลที่โค้ดในแล็บเขียน `paragraph.runs[0].bold = True` แทนที่จะตั้งที่ตัว paragraph ตรง ๆ

6.3 บทเรียนจากบัก `0_int`

บักนี้ต่างจากบักในสัปดาห์ก่อน ๆ ตรงที่เป็น syntax error ซึ่ง Python ตรวจพบตั้งแต่ตอนแปลงไฟล์ ยังไม่ทันรันบรรทัดใดเลย ทำให้ import ไฟล์ไม่ได้ทั้งไฟล์ แม้ฟังก์ชันอื่นในไฟล์จะเขียนถูกต้องก็ใช้ไม่ได้ ข้อดีคือพบง่ายเพราะพังทันที ต่างจากบักตรรกะที่โปรแกรมรันผ่านแต่ผลลัพธ์ผิด ซึ่งอันตรายกว่ามากเพราะอาจไม่มีใครสังเกตเห็น

7. บทสรุป

แล็บนีต่อยอดงานอัตโนมัติจากสเปรดชีตมาสู่เอกสาร ซึงเป็นงานทีพบบ่อยในทีทำงานจริง เช่น การรวมเอกสารหลายไฟล์ การดึงข้อมูลจากรายงาน PDF มาสรุป หรือการสร้างรายงาน Word อัตโนมัติจากข้อมูลดิบ จุดสำคัญทีได้เรียนรู้คือการแยกโมดูลตามประเภทงาน (PDF แยกจาก Word) ซึงทำให้โค้ดอ่านง่ายและนำไปใช้ซ้าได้ และการตรวจสอบผลลัพธ์จริงทุกครัง เช่นการเทียบจำนวนหน้าทีนับได้กับจำนวนหน้าจริงของไฟล์ต้นทาง

อ้างอิง: เอกสารประกอบการสอน "Week11_LabX BASIC: Automating PDFs & Word Documents"

หมายเหตุ: ไฟล์ PDF ดังต้นทัง 3 ไฟล์นำมาจาก GitHub ของผู้สอน ส่วนภาพในรายงานเรนเดอร์จากไฟล์ผลลัพธ์จริงทีโปรแกรมสร้างขึน (ไม่ใช่ภาพจับหน้าจอ OS โดยตรง)